SMU SINGAPORE MANAGEMENT UNIVERSITY | School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

SMU SINGAPORE MANAGEMENT UNIVERSITY
School of Computing and Information Systems

Computational Thinking with Python

Jean Chen **James Koh**

{jeanchen, jameskohby}@smu.edu.sg

Course Introduction

Why this course

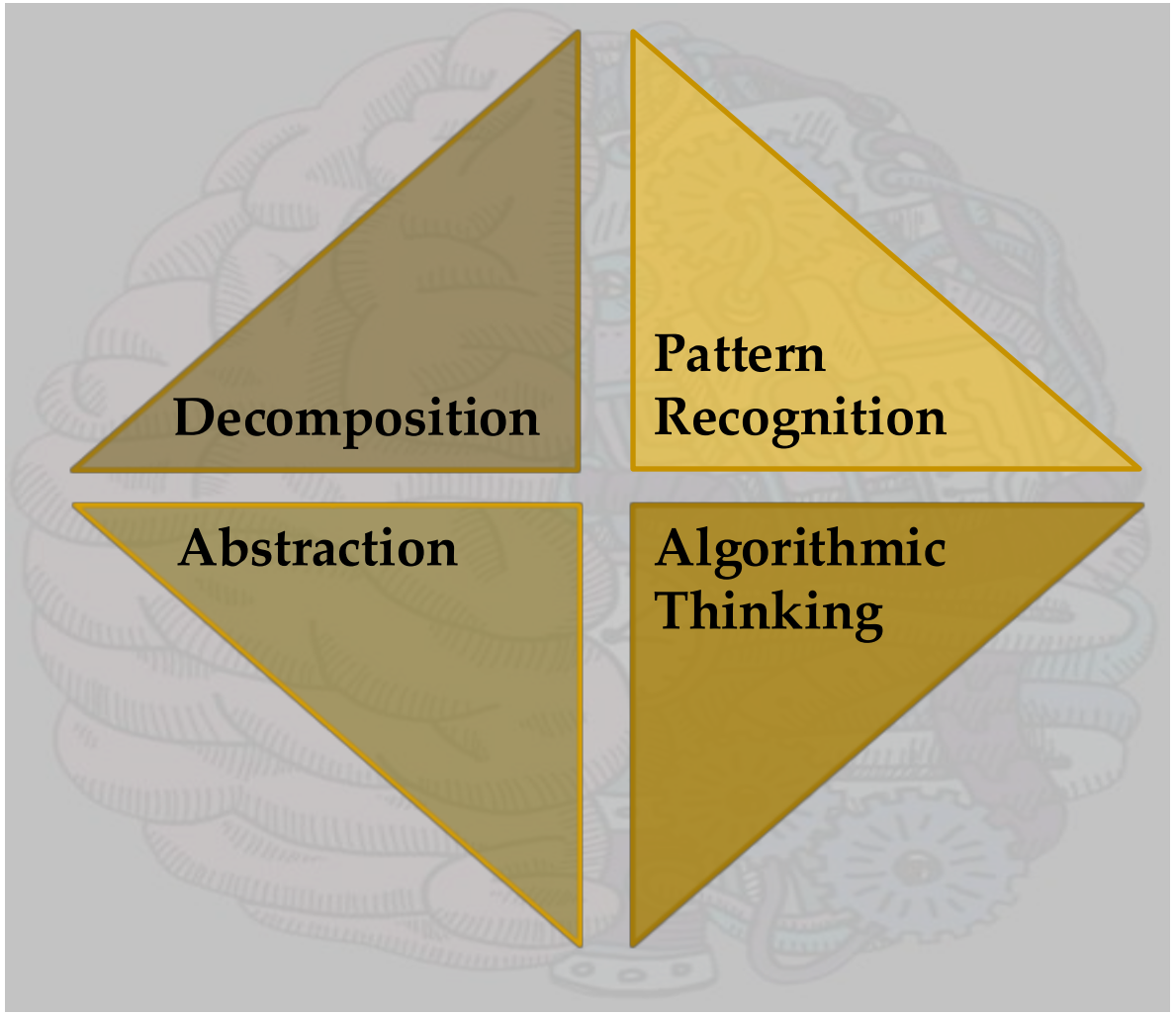
- This course is revamped from the previous course Python programming and data analysis, and now serves as a core course for MITB.

- Reasons for the revamp:
  - Too much content, as from students' feedback
  - Design for all MITB students
  - More importantly, computational thinking!

- What is the difference between programming and computational thinking?

© MITB, SMU

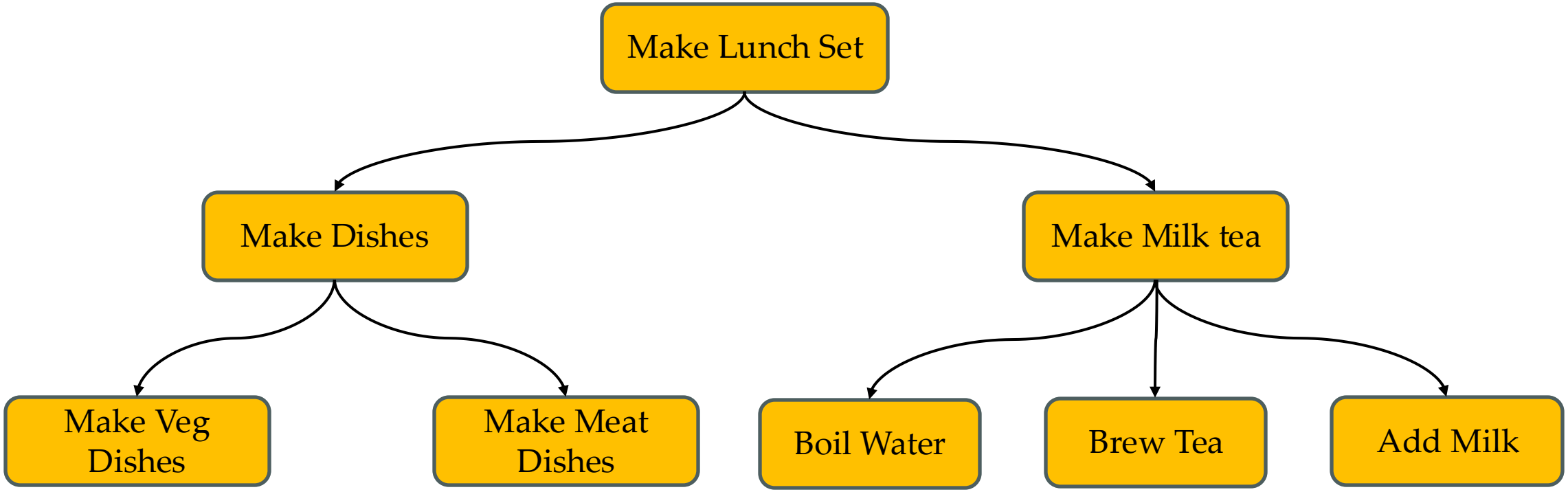
Computational Thinking



© MITB, SMU

Computational Thinking

- Decomposition



© MITB, SMU

Computational Thinking

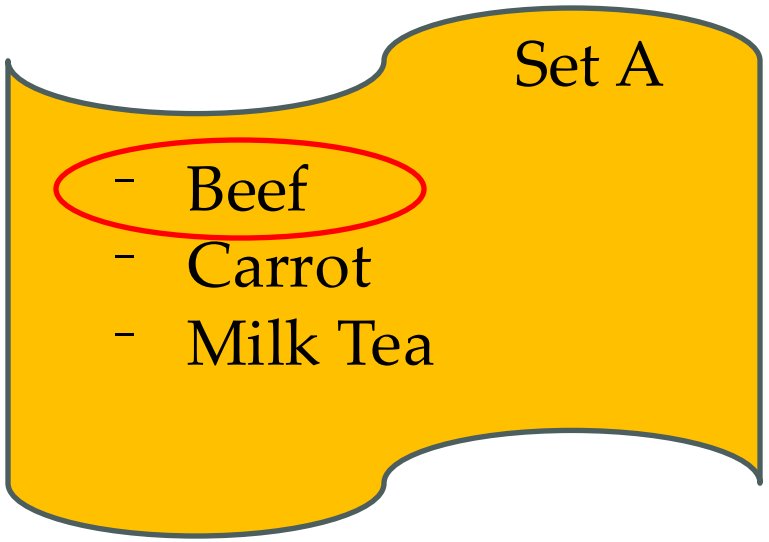
- Pattern Recognition

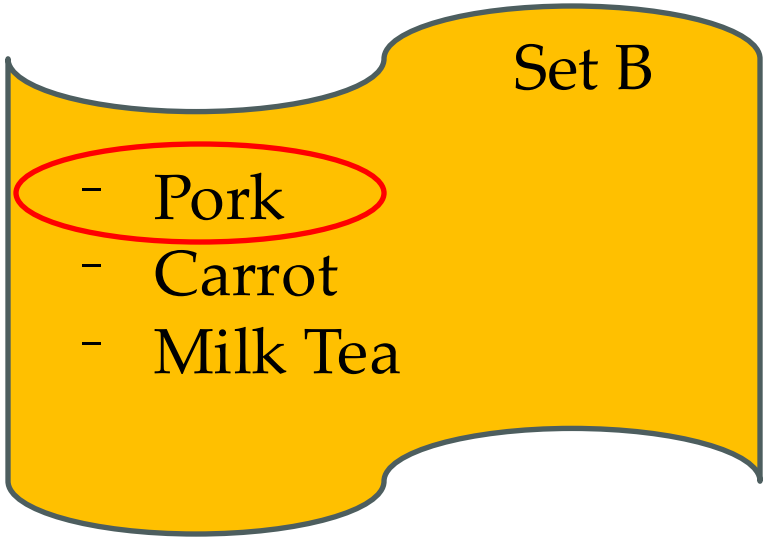
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
Beef Carrot	Beef Carrot	Pork Carrot	Pork Carrot	Pork Carrot	Pork Carrot	1 Pork Carrot
2 Beef Carrot	3 Beef Carrot	4 Pork Carrot	5 Pork Carrot	6 Pork Carrot	7 Pork Carrot	8 Pork Carrot
9 Beef Carrot	10 Beef Carrot	11 Pork Carrot	12 Pork Carrot	13 Pork Carrot	14 Pork Carrot	15 Pork Carrot

© MITB, SMU

Computational Thinking

- Abstraction: Focus on the important part



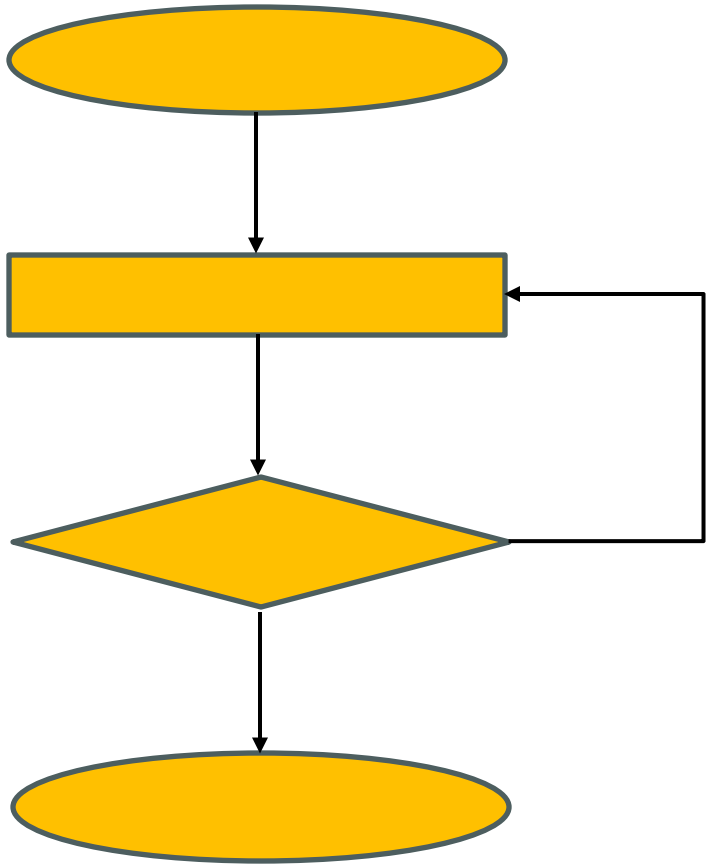


- To determine whether the dish is from set A or set B, we only need to know the type of meat.

© MITB, SMU

Computational Thinking

- Algorithmic Thinking: Step-by-step instructions



© MITB, SMU

Learning objectives

- Understand the elements and concepts of computational thinking
- Apply computational thinking in solving computational problems
- Be proficient in Python
- Remember, in this course, Python is like a sturdy tool for developing computational thinking.
- You will possess computational thinking skills even if you switch to another programming language

© MITB, SMU

Pseudocode ≈ Python


```
BEGIN
    INPUT n                 // Input the array size
    CREATE arr[n]           // Input the array elements
    FOR i FROM 0 TO n-1 DO
        INPUT arr[i]
    END FOR
    SET smallest TO arr[0] // Init smallest to 1st element
    FOR i FROM 1 TO n-1 DO // Loop through the array
        IF arr[i] < smallest THEN   // Update smallest if
            SET smallest TO arr[i]  // a smaller element exists
        END IF
    END FOR                 // Print the smallest element
    PRINT smallest
END
```

```
# Input the array size
n = int(input("Enter the number of elements: "))
# Input the array elements
arr = []
for i in range(n):
    element = int(input(f"Enter element {i+1}: "))
    arr.append(element)
smallest = arr[0] #Init smallest to 1st elem
for i in range(1, n):
    # Update smallest if a smaller element exists
    if arr[i] < smallest:
        smallest = arr[i]
# Print the smallest element
print("The smallest element is:", smallest)
```


© MITB, SMU

Schedule of Topics

Week 1 (from 24 Aug)	• Python Basics	
Week 2 (from 31 Aug)	• Decomposition	
Week 3 (from 7 Sep)	• Pattern Recognition	
Week 4 (from 14 Sep)	• Search	
Week 5 (from 21 Sep)	• Abstraction and Decomposition: Functions	Assignment Released
Week 6 (from 28 Sep)	• Abstraction: Object-Oriented Programming	
	Recess Week	
Week 7 (from 12 Oct)	• Regular Expressions	Assignment Due
Week 8 (from 19 Oct)	• Bit Operations	Timed Assignment (24 Oct 3.30pm to 4.30pm)
Week 9 (from 26 Oct)	• Applications of Computational Thinking	
Week 10 (from 2 Nov)	• Revision	
	• Final Exam	12 Nov 19:30-21:30



Key for assignment.

Saturday afternoon

Thu

© MITB, SMU

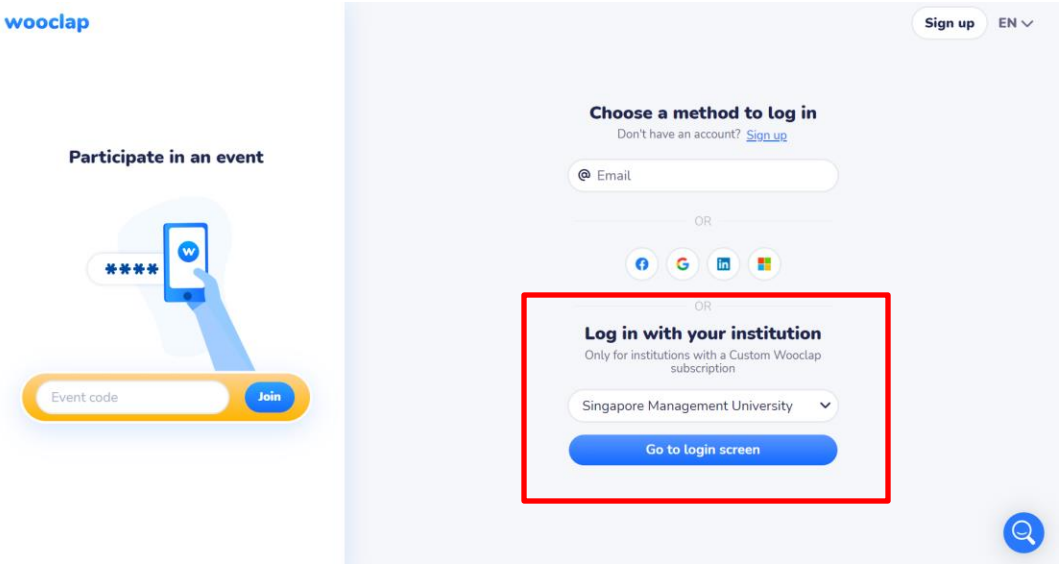
Assessments

- Class Attendance – 10% → Wooclap
- In-class Quizzes – 15%
- Take-home Assignment – 15%
- Timed Assignment – 15%
- Final Exam – 45%

© MITB, SMU

Class Attendance – 10%

- Login Wooclap with your **SMU account**
- Participate in Wooclap quizzes by entering the event code
- 2 min per quiz question (simulating the final exam :D) → clock questions can already, no need to save right or wrong
- Get 1 mark for each event, there are 10 in total



© MITB, SMU

In-class Quizzes – 15%

- 10-minute MCQs on elearn
- There will be 8 quizzes in total → week 2-9, 1 each week
- Each quiz will cover materials taught in the previous weeks
- The top 5 scores will be taken, and each contributes 3% to the final grade
- 5 x 3% = 15%

if cannot make it, email James & tell him that you want to attend another day's class

© MITB, SMU

Assignments – 15%

- Don't cheat!
  - Discussion on implementation is NOT allowed.
- Do it yourself!
- Have fun!

© MITB, SMU

In-class Timed Assignments – 15%

- 24 Oct (Saturday) 3.30pm to 4.30pm
- Scope: Anything within Lectures 1 to 5

MCQ ?

- no practice question
- can refer to code gone through in class & coding questions -

© MITB, SMU

Final Exam – 45%

- Summative evaluation of all the topics that have been covered throughout the term
- Closed-book
- Cheat sheet allowed (1 piece of A4 paper, double sided) → no cut & paste.
- Calculator allowed
- Lockdown browser
- Date & Time: 12 Nov 19:30-21:30

- no ide.

© MITB, SMU

Academic Integrity

- All acts of academic dishonesty (including, but not limited to, plagiarism, cheating, fabrication, facilitation of acts of academic dishonesty by others, unauthorized possession of exam questions, or tampering with the academic work of other students) are serious offences.
- All work (whether oral or written) submitted for purposes of assessment must be the student's own work.
- Penalties for violation of the policy range from zero marks for the component assessment to expulsion, depending on the nature of the offense.
- When in doubt, students should consult the instructors of the course.
- Details on the SMU Code of Academic Integrity may be accessed at

  smu.sharepoint.com/sites/oasis/SitePages/DOS-WKLSWC/UCSC.aspx

© MITB, SMU

Installation & File Opening

Prepare your computer for this course

- •Getting Python to run on your laptop
- –(Windows) https://medium.com/mitb-for-all/setting-up-visual-studio-28d7aed19fcd
- –(Mac) https://medium.com/mitb-for-all/setting-up-visual-studio-on-your-mac-1bf102bcabba

© MITB, SMU

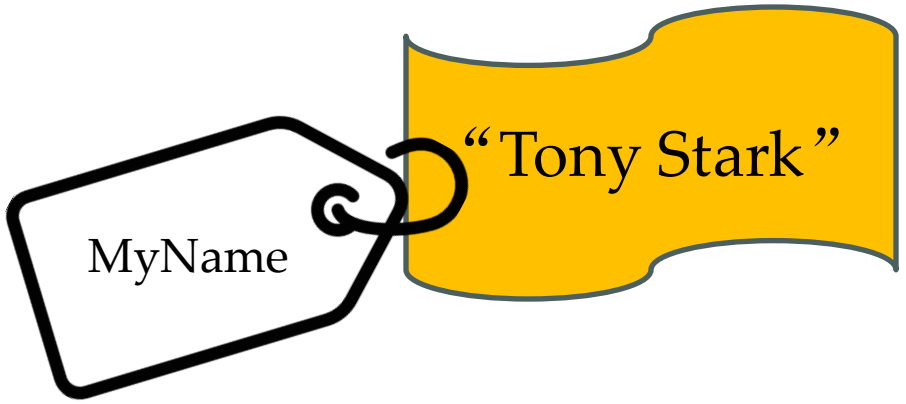
Basic Data Types

Basic Python variables

- Variables & Assignment

```
>>> MyName = ‘Tony Stark'
>>> print(MyName)
```

```
>>> print('MyName')
```



- Exercise: *Variables*
- Exercise: *Comments*

© MITB, SMU

Basic Python variables

- Booleans

```
>>> a = True
>>> print(type(a))   <class 'bool'>
>>> print(a)
```

- Floats

```
>>> a = 20.01
>>> print(type(a))   <class 'float'>
>>> print(a)
```

Python is a dynamically-typed language.

- Integers

```
>>> a = 20
>>> print(type(a))   <class 'int'>
>>> print(a)
```

- Strings

```
>>> a = 'the "art" of programming'
>>> print(type(a))   <class 'str'>
>>> print(a)
```

- Exercise: *Basic data types*

© MITB, SMU

Boolean variables

- What are the values of 'True' and 'False'?

```
>>> bin(True) # True = 1
'0b1'
>>> bin(False) # False = 0
'0b0'
```

- Type casting
  1. `int` cast to integers
  2. `float` cast to the float type
  3. `str` cast to the string
  4. `bool` cast to Boolean variables

```
>>> int(-1.99)
-1
>>> int('fedcba', 16)
16702650
>>> float('0.99999999999999999')
1.0
>>> str(2147483647)
'2147483647'
>>> bool(-1)
True
```

- Exercise: *Type casting*

© MITB, SMU

There is no string without encoding

no need in cheatsheet

base 16.

I just need to know how to map ⇒ ord, hex char, dec

- Strings are a sequence of characters

  If we can encode each character, we can encode the whole string

ASCII Table

Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char	Dec	Hex	Oct	Char
0	0	0		32	20	40	[space]	64	40	100	@	96	60	140	`
1	1	1		33	21	41	!	65	41	101	A	97	61	141	a
2	2	2		34	22	42	"	66	42	102	B	98	62	142	b
3	3	3		35	23	43	#	67	43	103	C	99	63	143	c
4	4	4		36	24	44	$	68	44	104	D	100	64	144	d
5	5	5		37	25	45	%	69	45	105	E	101	65	145	e
6	6	6		38	26	46	&	70	46	106	F	102	66	146	f
7	7	7		39	27	47	'	71	47	107	G	103	67	147	g
8	8	10		40	28	50	(	72	48	110	H	104	68	150	h
9	9	11		41	29	51	)	73	49	111	I	105	69	151	i
10	A	12		42	2A	52	*	74	4A	112	J	106	6A	152	j
11	B	13		43	2B	53	+	75	4B	113	K	107	6B	153	k
12	C	14		44	2C	54	,	76	4C	114	L	108	6C	154	l
13	D	15		45	2D	55	-	77	4D	115	M	109	6D	155	m
14	E	16		46	2E	56	.	78	4E	116	N	110	6E	156	n
15	F	17		47	2F	57	/	79	4F	117	O	111	6F	157	o
16	10	20		48	30	60	0	80	50	120	P	112	70	160	p
17	11	21		49	31	61	1	81	51	121	Q	113	71	161	q
18	12	22		50	32	62	2	82	52	122	R	114	72	162	r
19	13	23		51	33	63	3	83	53	123	S	115	73	163	s
20	14	24		52	34	64	4	84	54	124	T	116	74	164	t
21	15	25		53	35	65	5	85	55	125	U	117	75	165	u
22	16	26		54	36	66	6	86	56	126	V	118	76	166	v
23	17	27		55	37	67	7	87	57	127	W	119	77	167	w
24	18	30		56	38	70	8	88	58	130	X	120	78	170	x
25	19	31		57	39	71	9	89	59	131	Y	121	79	171	y
26	1A	32		58	3A	72	:	90	5A	132	Z	122	7A	172	z
27	1B	33		59	3B	73	;	91	5B	133	[	123	7B	173	{
28	1C	34		60	3C	74	<	92	5C	134	\	124	7C	174	|
29	1D	35		61	3D	75	=	93	5D	135	]	125	7D	175	}
30	1E	36		62	3E	76	>	94	5E	136	^	126	7E	176	~
31	1F	37		63	3F	77	?	95	5F	137	_	127	7F	177	

- How to convert a character to a number?
- Exercise: *String encoding*

en.wikibooks.org/wiki/A-level_Computing/OCR/Unit_1.4.1_Data_Types#ASCII

© MITB, SMU

Wooclap!

What is the output of the below code?

```
a = '610'
print(int(a) * 2, int(a * 2))
```



a. 1220 610610
b. 610610 1220
c. 1220 1220
d. 610610 610610

Python Operators

Python operators

Four categories of Python operators

- Arithmetic operators
- Logical operators
- Assignment operators
- Comparison operators

Other operators – to be introduced in the future lectures

- Identity operators
- Membership operator
- Bitwise operator

© MITB, SMU

Arithmetic operators

Boring ones

- `+` addition
- `-` subtraction
- `*` multiplication
- `/` division

Guess what are

`+=` `-=`

`*=` `/=` ?

```
>>> 5 + 3
8
>>> 5.0 + 3
8.0
>>> 5 * 3
15
>>> 5 / 3
1.6666666666666667
>>> n = 5
>>> n += 2
>>> n
7
```


© MITB, SMU

Arithmetic operators

Interesting ones

- // integer division
  a.k.a. Euclidean division
- % modulo
- ** exponent
- e power of 10

- Exercise: *Arithmetic operators*

```
>>> 5 / 3
1.6666666666666667
>>> 5 // 3
1
>>> 5 % 3
2
>>> 5 ** 3
125
>>> 5 / 300000
1.6666666666666667e-05
>>> 5e3
5000.0
```


© MITB, SMU

Logical operators

- Boolean operators
  1. **`not` : unary operator**
  2. `and` : binary operator
  3. `or` : binary operator

```
>>> not False
True
>>> a = 9
>>> a >= 9
True
>>> not a < 9
True
```

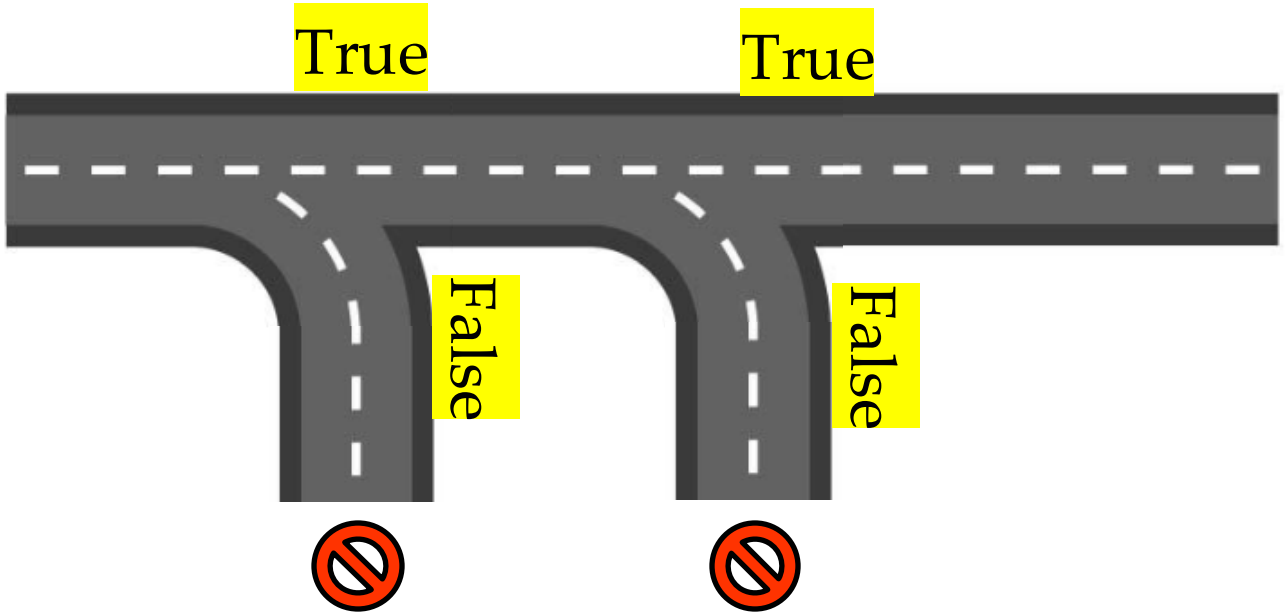

© MITB, SMU

Logical operators

- Boolean operators
  1. `not` : unary operator
  2. **`and` : binary operator**
  3. `or` : binary operator

AND table	
True and True	True
True and False	False
False and True	False
False and False	False

```
>>> True and False
False
>>> True and True
True
```



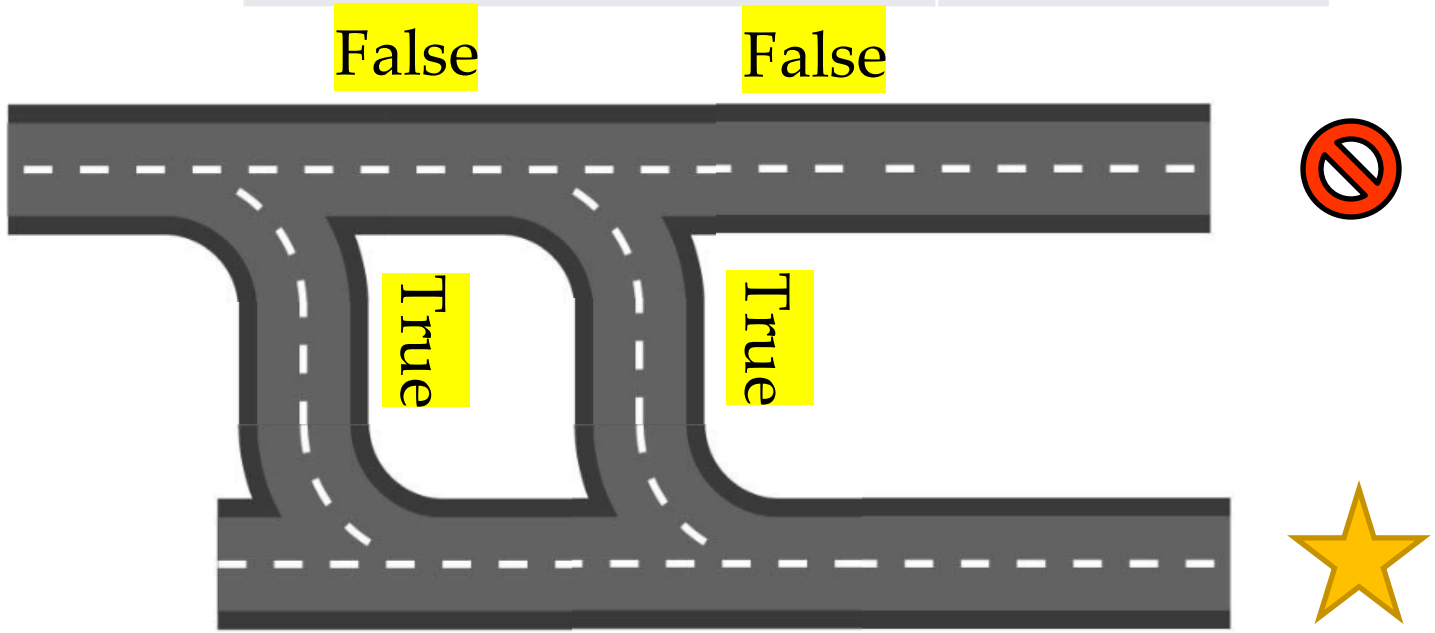
© MITB, SMU

Logical operators

- Boolean operators
  1. `not` : unary operator
  2. `and` : binary operator
  3. **`or` : binary operator**

OR table	
True or True	True
True or False	True
False or True	True
False or False	False

```
>>> False or False
False
>>> False or True
True
```



© MITB, SMU

Assignment operators

The one and only one:

- `=` assignment

- Exercise: *Assignment operators*

Compound operators:

- `+=` add and assign
- `-=` subtract and assign
- `*=` multiply and assign
- `/=` divide and assign
- `//=` integer divide and assign
- `%=` modulo and assign
- `**=` exponent and assign

© MITB, SMU

Comparison operators

- `==` equal
- `!=` not equal
- `>` greater than
- `<` less than
- `>=` greater than or equal
- `<=` less than or equal

Comparisons on numerical values (integers and floats) are straightforward.

```
>>> a = True
>>> b = False
>>> a > b
True
```

Boolean comparisons

```
>>> a = 'a'
>>> b = 'A'
>>> a < b
False
```

String comparisons

- Exercise: *Comparison operators*

© MITB, SMU

Wooclap!

What is the output of the below code?

```
print('ab' < 'ac' and '0c' < '1a')
```

a. True
b. False

String operations – concatenations & repetitions

String operations - much more than operations for other data types

- **Concatenations (+) and repetitions (*)**
- Index operations
- Search operations
- Print operations

```
>>> 'news' + 'paper' + 'man'
'newspaperman'
>>> s = 'eigen'
>>> s += 'value'
>>> s
'eigenvalue'
>>> '-' * 10
----------
```

+ for concatenations

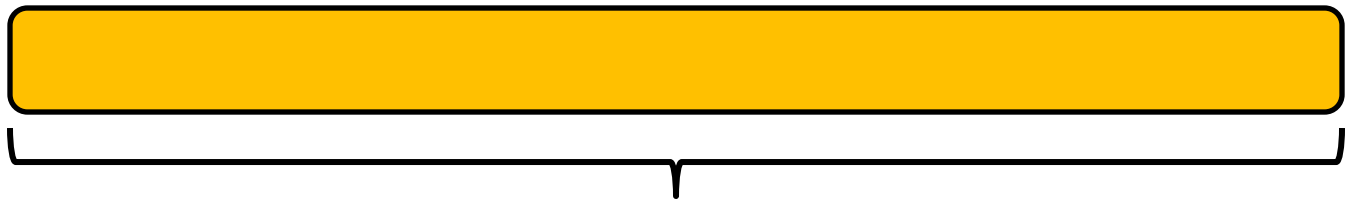
+= : concatenate and assign

***** for repetitions

© MITB, SMU

String operations – index operations

- `len(s)`: gives the length of string `s` in characters



- Indices start from 0, last index at `len()-1`

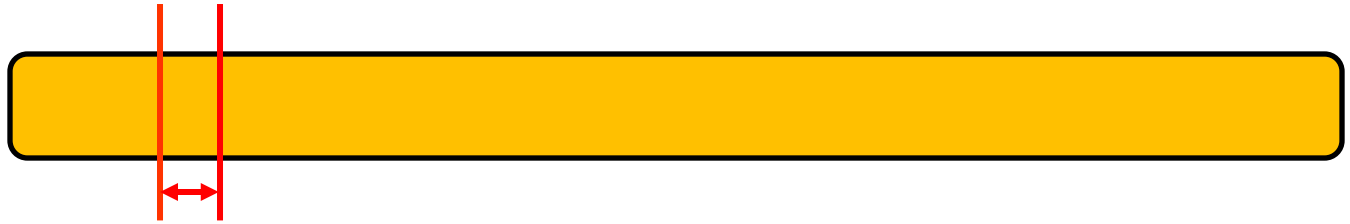
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

```
>>> s = 'machine learning'
>>> len(s)
16
```


© MITB, SMU

String operations – index operations

- Slice one character
  - `string[index]`
  - e.g., s[2]



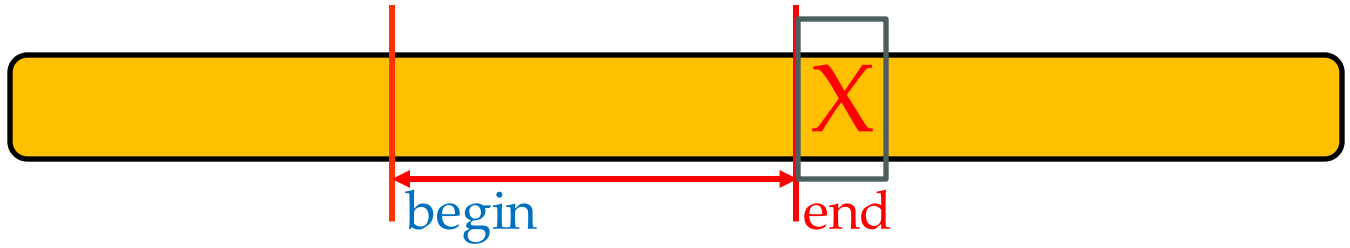
```
>>> s = 'machine learning'
>>> s[2]
'c'
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

© MITB, SMU

String operations – index operations

- Slice substrings
  - `string[begin:end]`: from index `begin` to index `end`,
    inclusive of `begin`, but exclusive of `end`



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

```
>>> s = 'machine learning'
>>> s[5:10]
'ne le'
```

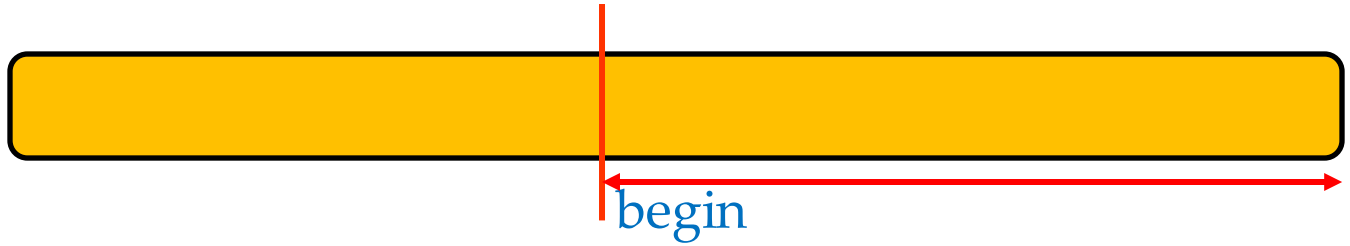
12345

subtract to get number of char returned : 10 - 5 = 5

© MITB, SMU

String operations – index operations

- Slice substrings
  - `string[begin:]`: to the end of the string if end is omitted
  - `string[begin:large_number]`: to the end of the string




```
>>> s = 'machine learning'
>>> s[8:]
'learning'
>>> s[8:100]
'learning'
```

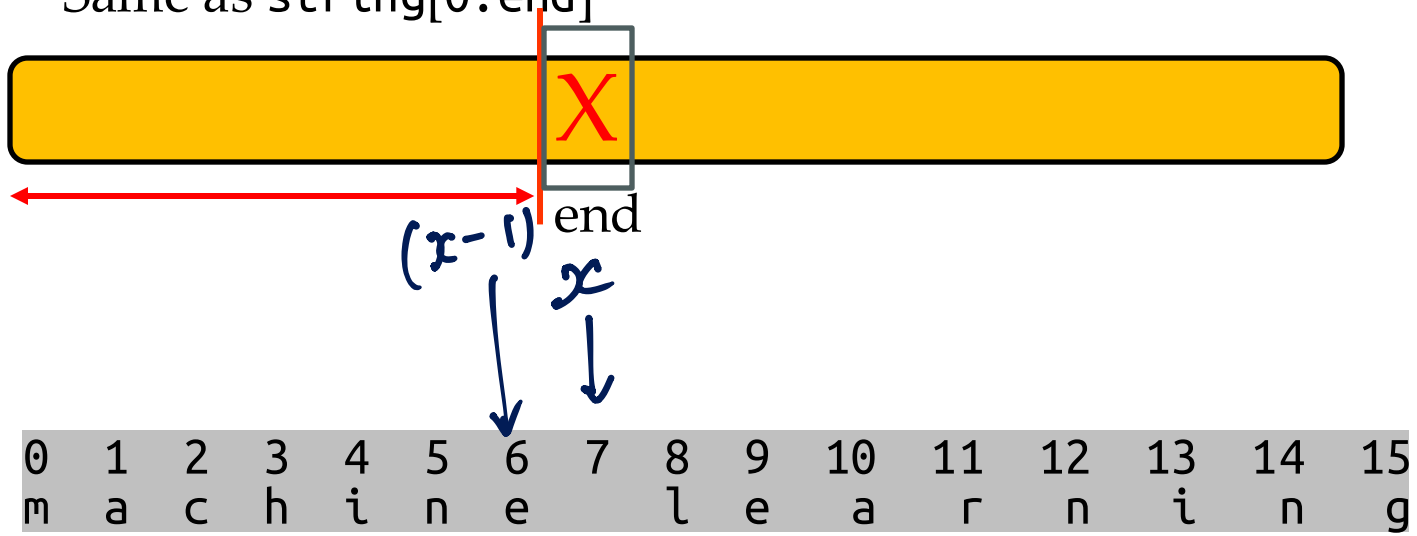
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

© MITB, SMU

String operations – index operations

- Slice substrings
  - `string[:end]`: from the beginning of the string if begin is omitted,

    character at end is excluded
  - Same as `string[0:end]`

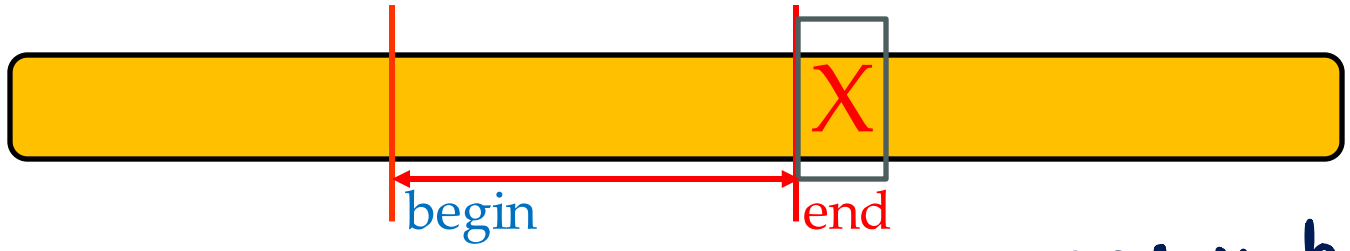



```
>>> s = 'machine learning'
>>> s[:7]
'machine'
>>> s[0:7]
'machine'
```


© MITB, SMU

String operations – index operations

- Slice substrings – negative index
  - `string[begin:end]`: from index `begin` to index `end`, inclusive of `begin`, but exclusive of `end`




```
>>> s = 'machine learning'
>>> s[-7:-3]
'earn'
>>> s[9:-3]
>>> s[-7:13]
```

pos index - len = neg index

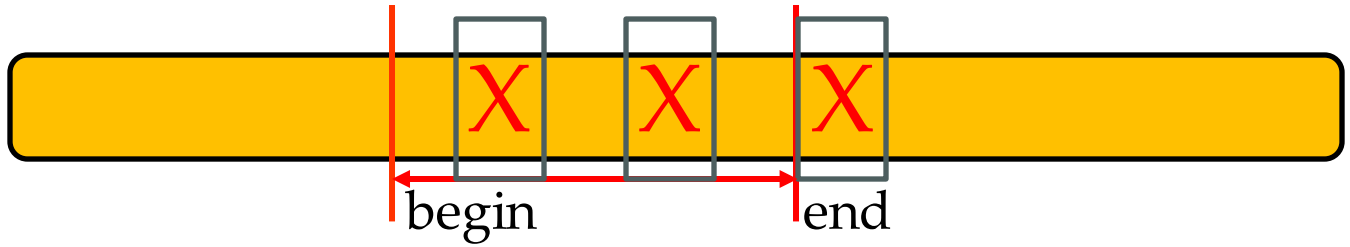
same

-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

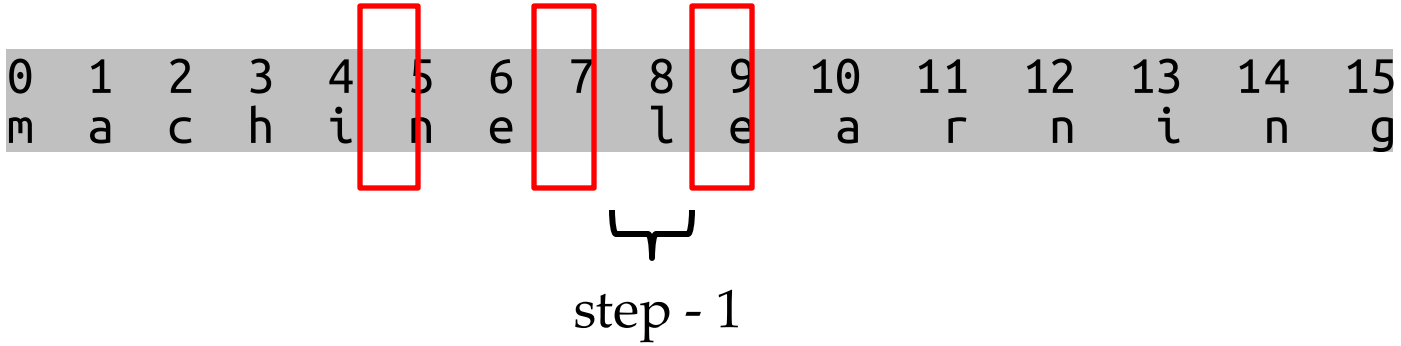
© MITB, SMU

String operations – index operations

- Slice substrings – with step (default = 1)
  - `string[begin:end:step]`: from index `begin` to index `end`, at a `step` size, inclusive of `begin`, but exclusive of `end`




```
>>> s = 'machine learning'
>>> s[5:10:2]
'n e'
```

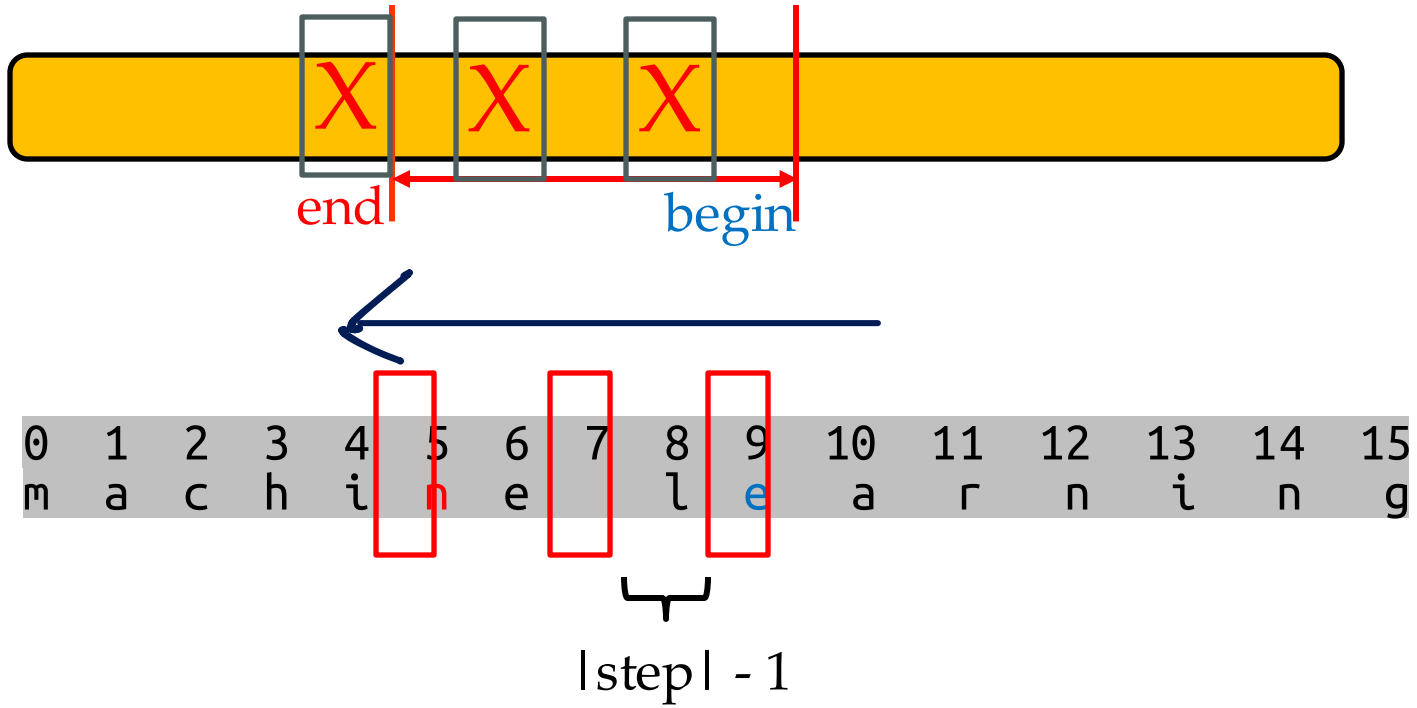


© MITB, SMU

String operations – index operations

- Slice substrings – with negative steps (from right to left)
  - `string[begin:end:step]`: from index `begin` to index `end`,

    at a `step` size, inclusive of `begin`, but exclusive of `end`



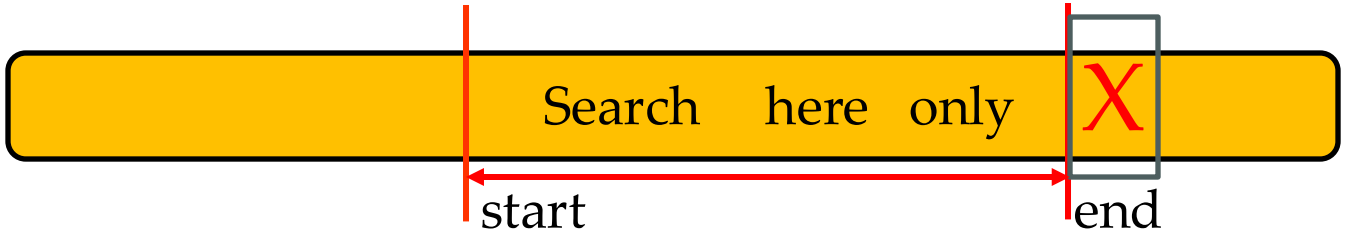

```
>>> s = 'machine learning'
>>> s[9:4:-2]
'e n'
```


© MITB, SMU

String operations – search operations

- `find` find the **starting** index of the **first** occurrence of substring
  - `string.find(substring,start,end)`
  - inclusive of `start`, but exclusive of `end`
  - `-1` represents not found, *not a negative index here*






```
>>> s.find('a')
1 → true, got a

>>> s.find('a', 4)
10 → gives 1st occurrence of a and ends.

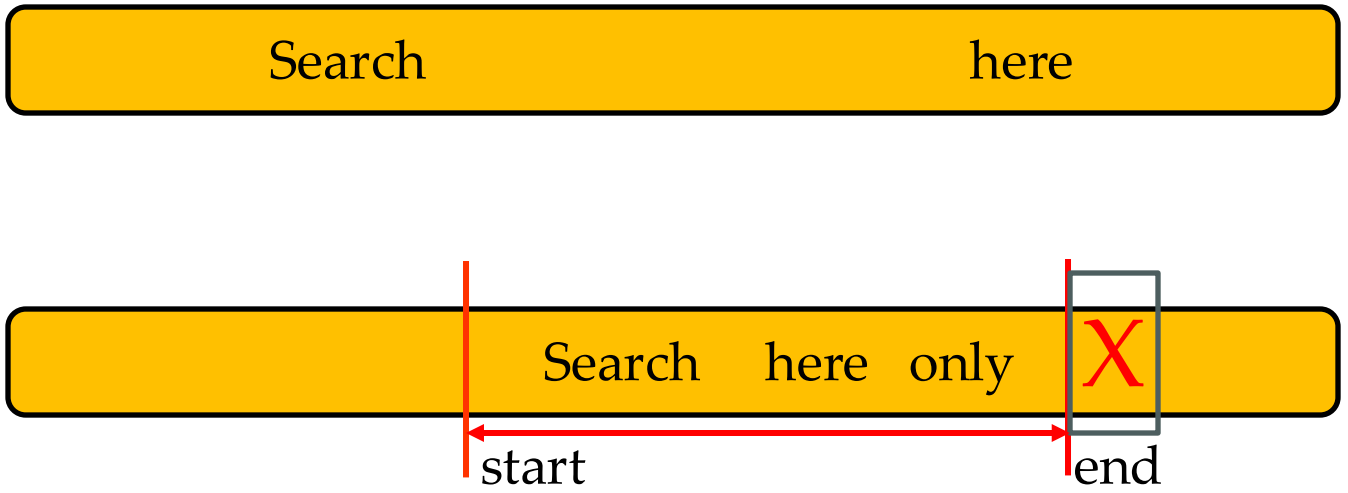
>>> s.find('a', 4, 10)
-1 → false, no a
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

© MITB, SMU

String operations – search operations

- `rfind` find the **starting** index of the **last** occurrence of substring
  - `String.rfind(substring,start,end)`
  - inclusive of `start`, but exclusive of `end`
  - `-1` represents not found, *not a negative index here*




```
>>> s.rfind('a')
10
>>> s.rfind('a', 4)
10
>>> s.rfind('a', 4, 10)
-1
```

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
m	a	c	h	i	n	e		l	e	a	r	n	i	n	g

© MITB, SMU

String operations – print operations

- Cast to `str` and print

```
>>> print(1234, 567.89)
    print("usually next line")
1234 567.89
usually next line


>>> print(1234, 567.89, sep = '-', end = '#’)
    print("usually next line")
1234-567.89#usually next line
```


© MITB, SMU

Wooclap!

What is the output of the below code?

```
s = "hello, world!"
s = s[::2] + s[1::2]
print(s)
```

a. hello, world!
b. hlo ol!el,wrd
c. hello, worlello, worl
d. hello, world!hello, world!

String operations – print operations

- Print with format()
  - {}

  Replace the string in order
  - {index}

  Replace the string with index
  - {name}

  Replace the string with variable name

```
>>> print('Program name: {} \nCourse name: {} \nCourse code: {}'.format('MITB', 'CT with Python', 'IS628'))
Program name: MITB
Course name: CT with Python
Course code: IS628
>>> print('Program name: {1} \nCourse name: {0} \nCourse code: {2}'.format('MITB', 'CT with Python', 'IS628’))
>>> print('Program name: {program} \nCourse name: {course} \nCourse code: {id}'.format(course = 'CT with Python', id = 'IS628', program = 'MITB'))
```


© MITB, SMU

String operations – print operations

- Print with format()
  - {?:specs}

  Format the ? (anything from previous slide, try it out!)

  according to the specs
  - Specs
    - d: decimal integer
    - f: float
    - s: string
    - 03d: 3 decimal places with leading 0
    - ^5d: 5 decimal places & centered
    - .6f: 6 places after decimal point

```
>>> print('{:s} is approximately {:03d}/{:<2d} = {:.2f}'.format('pi', 22, 7, 3.1415926))
pi is approximately 022/7  = 3.14
>>> print('{0:s} is approximately {1:^5d}/{2:^5d} = {3:.6f}'.format('pi', 355, 113, 3.1415926))
pi is approximately  355 / 113  = 3.141593
```

- Exercise: *String operators*

© MITB, SMU

Wooclap!

Given the below code, what should be filled into the blank to get the output "123.457"?

```
s = 123.4567890
print("{: _________ }".format(s))
```

a. .2f
b. .3f
c. .2d
d. .3d

Control Flow

Conditionals: if-else switch

Simple if-else:

- `if` clause is a Boolean statement containing some variables, which evaluates as True or False
- When it evaluates True, the operations on the if branch will be activated
- When it evaluates False, the operation on the else branch will be activated

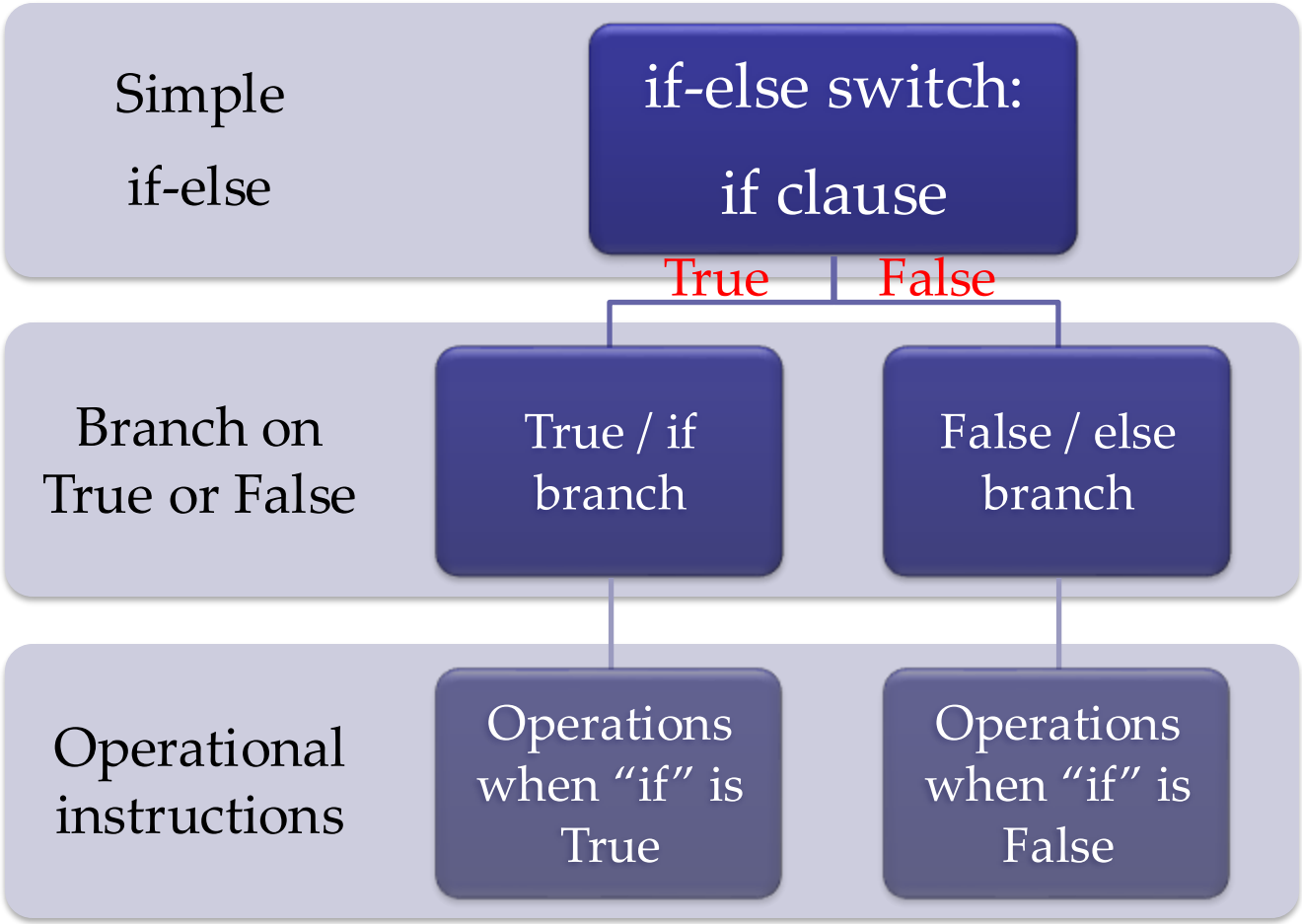
```
if a test condition on variable x:
    do something when the test condition is true
else:
    do something when the test condition is false
```


© MITB, SMU

Conditionals: if-else switch

Simple if-else:

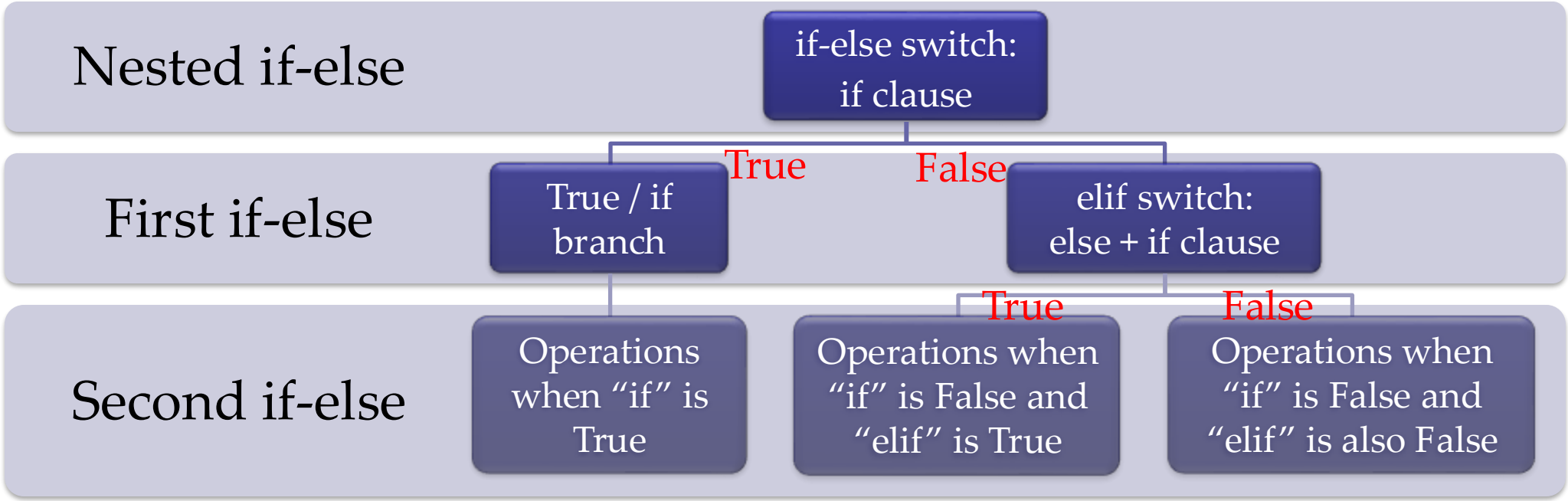
- `if` clause is a Boolean statement containing some variables, which evaluates as True or False
- When it evaluates True, the operations on the if branch will be activated
- When it evaluates False, the operation on the else branch will be activated



© MITB, SMU

elif = else + if

- Another if-switch can be appended at either branch
- When it is appended at False branch, we can use `elif` followed by a Boolean statement, and split into another two branches from `elif`



© MITB, SMU

elif example


```
>>> i = int(input())
... if i % 2 == 0:
...     print('this is an even number')
... else:
...     print('this is an odd number')
```

```
>>> i = int(input())
... if i % 2 == 0:
...     print('this is an even number')
... elif i % 3 == 0:
...     print('this is a multiple of 3')
... else:
...     print('this is a prime number')
```

- Exercise: *Conditionals*

© MITB, SMU

Exercises

Exercise 1: Calculate BMI

Body Mass Index is a simple calculation using a person's height and weight.
The formula is $BMI = kg/m^2$ where kg is a person's weight in kilograms and m^2 is their height in meters squared.
A BMI of 25.0 or more is overweight, while the healthy range is 18.5 to 24.9.
Write a BMI calculator to understand your expected BMI weight status.

© MITB, SMU

Exercise 2: Calculate Carbon

- When you execute a heavy computational workload in the cloud, you may notice that the final environmental impact exceeds the raw energy used by the CPU. This is primarily due to two environmental multipliers: Grid Carbon Intensity and the data center's Power Usage Effectiveness (PUE). While the server's base energy is simply its hardware power (kW) multiplied by its runtime (hours), the PUE typically adds a 20% overhead for facility cooling. Some unoptimized codebases even impose an "inefficient algorithm surcharge" that drastically increases the total runtime.
- There are two types of cloud regions you can deploy to:
- **Clean Region** (Carbon Intensity is 0.030 kg CO_2/kWh)
- **Standard Region** (Carbon Intensity is 0.450 kg CO_2/kWh)
- Inefficient Penalty (Increases base runtime by a factor of 10) – (Y/N)
- Write a carbon footprint calculator for estimating the actual emissions.

© MITB, SMU

Exercise 3: Shorten a Word

Letters between the first and last letters of a word may be replaced by the number of letters omitted.

For example, the word "internationalization" can be abbreviated by replacing the eighteen middle letters ("nternationalizatio") with "18", leaving "i18n".

These word shortenings are sometimes called numerical contractions.

Examples of numerical contractions include:

- s5n – shorten
- l10n – localization
- i18n – internationalization

Write a generator to help generate the numerical contractions.

© MITB, SMU

Questions?

Thank You



Questions?

Thank You

